UNIDESK DATABASE SYSTEM

3-Person Collaborative Git Workflow & Instructor Compliance Playbook

A rigorous guide to feature branching, linear commit footprints, branch protection, and equal grading split

> **INSTRUCTOR COMPLIANCE NOTICE**
> This repository is governed by strict academic contribution auditing. Commits must be iterative, frequency-balanced across all three contributors, scoped to designated modules, and merged strictly via reviewed Pull Requests. Direct pushes to main or bulk single-commit uploads violate grading policy and result in zero-grade penalties.

Ø=Üe Team Role Allocation & Ownership Matrix

To ensure all three team members receive full contribution grades, each member strictly owns one module directory. Cross-module edits must be coordinated via linked Pull Requests.

Team Member	Assigned Module	Directory Scope	Sample Branch Prefix
Person A	Frontend Client Application	frontend/	feature/fe-*
Person B	Backend API & DB Middleware	backend/	feature/be-*
Person C (Admin)	Database Architecture & Docs	database/ & docs/	feature/db-*

Ø=ÜÁ 1. Repository Structure & File Governance

The repository is organized into isolated functional tiers. Every member must respect boundaries to prevent merge collisions.

```
unidesk-database-system/
% % %  .gitignore                    # Root level ignore (REQUIRED - stage first)
% % %  README.md                     # Setup instructions & member ownership matrix
% % %  docs/                         # Person C (ERD diagrams, DB architecture specs)
%    % % %  DATABASE_SCHEMA_DESIGN.md
%    % % %  unidesk_schema.drawio
%    % % %  DRAWIO_IMPORT_GUIDE.md
% % %  database/                     # Person C (SQL DDL/DML, triggers, routines, RBAC)
%    % % %  01_schema_3nf.sql
%    % % %  02_triggers.sql
%    % % %  03_procedures_functions.sql
%    % % %  04_rbac.sql
%    % % %  05_seed_data.sql
% % %  backend/                      # Person B (Node.js/Express API & connection pool)
%    % % %  package.json & package-lock.json
%    % % %  .env.example               # Sanitized template (NEVER real credentials)
%    % % %  src/ (server.js, db.js, routes/)
% % %  frontend/                     # Person A (React/Vite single-page application)
    % % %  package.json & package-lock.json
    % % %  index.html, vite.config.js
    % % %  src/ (App.jsx, components/)
```

File / Artifact	Status	Inspection Criterion
Root .gitignore	REQUIRED	Prevents node_modules, .env, and dist from contaminating history.
Root README.md	REQUIRED	Documents setup, prerequisites, and explicit 3-member contribution matrix.
backend/.env.example	REQUIRED	Proves secret governance. Contains dummy keys (DB_HOST, DB_USER).
package.json & lock	REQUIRED	Committed in both frontend/ and backend/ for deterministic builds.
database/*.sql scripts	REQUIRED	Direct proof of 3NF schema, procedures, triggers, RBAC, and seed dataset.
docs/ ERD & Schemas	REQUIRED	Visual proof of relational integrity (PKs, FKs, Cardinality).
.env files (secrets)	FORBIDDEN	Credential leakage penalty. Zero grade on security audit.
node_modules/ or dist/	FORBIDDEN	Bulk file dumping penalty. Bloats repository index.

Ø=Ý 2. Fair Contribution Split & Identity Authentication

GitHub calculates contributions in Insights > Contributors based strictly on the email associated with each commit. If a team member commits with an unverified or local machine email, their commits will show as an unlinked gray avatar, resulting in zero contribution points.

Machine Configuration (Run once on each laptop before committing):

Person A (Frontend Lead):

```
git config --global user.name "Person A Full Name"
git config --global user.email "person-a-email@university.edu"
```

Person B (Backend Lead):

```
git config --global user.name "Person B Full Name"
git config --global user.email "person-b-email@university.edu"
```

Person C (Database Lead & Admin):

```
git config --global user.name "Person C Full Name"
git config --global user.email "person-c-email@university.edu"
```

> **EMAIL VERIFICATION RULE**
> The email entered in git config user.email MUST exactly match an email added and verified under GitHub Profile > Settings > Emails. If you use a university email, verify both your university email and GitHub primary email.

Initial Repository Seeding (Person C / Admin only):

```
cd unidesk-database-system
git init -b main
git add .gitignore README.md
git commit -m "chore(repo): initialize repository structure and gitignore governance"
git remote add origin https://github.com/<org-or-user>/unidesk-database-system.git
git push -u origin main
```

Once Person C pushes the initial seed commit, Person A and Person B clone the repository:

```
git clone https://github.com/<org-or-user>/unidesk-database-system.git
cd unidesk-database-system
```

Ø=Þáþ 3. Branch Protection Setup & Repository Governance

To comply with the non-negotiable rule ("Pushing directly to main is prohibited. The core branch must be locked"), Person C must configure GitHub Branch Protection immediately.

- **Administrator Role:** Designate Person C as Repository Administrator (Repository Owner).
- **Collaborator Access:** Add Person A and Person B under Settings > Collaborators with Write permissions.

Step-by-Step GitHub Web UI Protection Settings:

- **Step 1:** In your GitHub repo, click Settings (top right tab) > Branches (left sidebar).
- **Step 2:** Click Add branch protection rule and set Branch name pattern to: main
- **Step 3:** Require a pull request before merging (Set required approvals to: 1).
- **Step 4:** Dismiss stale pull request approvals when new commits are pushed.
- **Step 5:** Require linear history (Eliminates messy merge bubbles; keeps tracking tree clean).
- **Step 6:** Do not allow force pushes (Prevents destructive git push --force).
- **Step 7:** Do not allow deletions (Prevents accidental branch removal).
- **Step 8:** Do not allow bypass of the above settings (Enforce rules for Administrators).

Equivalent GitHub CLI (gh) Command:

```
gh api --method PUT /repos/:owner/:repo/branches/main/protection \
  --input - <<< '{
    "required_status_checks": null,
    "enforce_admins": true,
    "required_pull_request_reviews": {
      "dismiss_stale_reviews": true,
      "require_code_owner_reviews": false,
      "required_approving_review_count": 1
    },
    "restrictions": null,
    "required_linear_history": true,
    "allow_force_pushes": false,
    "allow_deletions": false
}'
```

Ø<ß? 4. Feature-Branch Workflow & Pull Request Pipeline

Direct pushes to main are strictly forbidden. Every feature, bug fix, or schema evolution must originate on an isolated side branch and be merged through a peer-reviewed Pull Request.

Standard Feature Branch Naming Syntax:

- **Frontend (Person A):** feature/fe-auth-ui, feature/fe-campus-map, feature/fe-visitor-routing
- **Backend (Person B):** feature/be-express-pool, feature/be-request-crud-api, feature/be-auth-middleware
- **Database (Person C):** feature/db-3nf-schema-v1, feature/db-audit-triggers, feature/db-stored-procedures

Command-by-Command Lifecycle (Example: Person C adding triggers):

```
# 1. Always pull latest main before starting
git checkout main
git pull origin main

# 2. Create and switch to your feature branch
git checkout -b feature/db-audit-triggers

# 3. Work inside your assigned folder & stage small atomic commits
git add database/02_triggers.sql
git commit -m "feat(db): implement audit trigger for requests status changes"

# 4. Make further iterative refinements
git add database/02_triggers.sql
git commit -m "fix(db): correct old-value null handling in relocation trigger"

# 5. Fetch and rebase against main before pushing to ensure linear graph
git fetch origin
git rebase origin/main

# 6. Push the feature branch to GitHub
git push -u origin feature/db-audit-triggers
```

Opening & Merging the Pull Request on GitHub:

- **Step 1:** Open the repo in GitHub. Click Compare & pull request on your pushed branch.
- **Step 2:** Title: feat(db): automated audit triggers for status & relocation changes.
- **Step 3:** Description: Fill out the brief PR template (Summary of Changes & Testing Steps).

• **Step 4:** Reviewers: Assign your teammates (Person A and Person B) on the right sidebar.
• **Step 5:** Teammate Review: Person A or B opens PR > Files changed > Review changes > Approve.
• **Step 6:** Merge: Click Rebase and merge (or Squash and merge).
• **Step 7:** Clean up: Click Delete branch on GitHub, then clean up locally: git checkout main && git pull origin main && git branch -d feature/db-audit-triggers.

' þ 5. Commit Hygiene & Avoiding the Bulk Drop Penalty

Commits must adhere to the Conventional Commits specification. A single massive commit containing an entire module is flagged as non-compliant.

Commit Message Anatomy: <type>(<scope>): <concise description>

• **Person A:** feat(fe): scaffold react application with vite and custom routing
• **Person A:** feat(fe): build campus directory hero with interactive search input
• **Person A:** feat(fe): implement password visibility toggle with eye-off icon
• **Person A:** feat(fe): create public visitor default interface and auth modal
• **Person B:** feat(be): initialize express server and configure cors middleware
• **Person B:** feat(be): setup mysql2 connection pool with environment variables
• **Person B:** feat(be): implement auth login endpoint with bcrypt verification
• **Person B:** feat(be): build service request crud endpoints with transaction support
• **Person C:** docs(erd): finalize 15-table 3nf relational schema architecture
• **Person C:** feat(db): define core tables, primary keys, and foreign key cascades
• **Person C:** feat(db): write stored procedure for appointment scheduling validation
• **Person C:** feat(db): establish rbac roles, table grants, and operational permissions

Staging Partial Work with Precision:

```
# Target specific files (NEVER run generic `git add .`)
git add frontend/src/App.jsx
git commit -m "feat(fe): add show-hide password toggle to login"

# Interactive hunk staging (stage parts of a modified file)
git add -p frontend/src/App.jsx
```

Emergency: What if you made a giant commit locally before pushing?

```
# Undo the single giant commit, but KEEP all modified files intact in working directory
git reset --soft HEAD~1

# Now break it into 4 separate logical commits:
git add backend/package.json backend/.env.example
git commit -m "chore(be): setup dependencies and environment configuration"

git add backend/src/config/db.js
git commit -m "feat(be): implement mysql connection pool manager"

git add backend/src/routes/auth.js
git commit -m "feat(be): build user authentication and password hashing logic"

git add backend/src/server.js
git commit -m "feat(be): assemble api router and health check listener"
```

Ø=ÜÊ 6. Multi-Week Cadence & Live Inspection Checklist

Distribute your Pull Requests evenly across project sprints to produce an organic, credible software engineering network graph.

Sprint / Week	Person C (Database Lead)	Person B (Backend Lead)	Person A (Frontend Lead)
Week 1 (Foundations)	PR #1: 3NF Relational Tables DDL	PR #2: Express Server & DB Pool	PR #3: Vite Setup & Design Tokens
Week 2 (Core Logic)	PR #4: Business Stored Procedures	PR #5: Office Directory Search API	PR #6: Campus Directory & Office Views
Week 3 (Integrations)	PR #7: Audit Triggers & RBAC Grants	PR #8: Booking & CRUD Endpoints	PR #9: Visitor Interface & Request Forms
Week 4 (Hardening)	PR #10: Seed Datasets & ERD Docs	PR #11: Input Validation & Error API	PR #12: Password Eye Toggles & Polish

Live Inspection Day Checklist (Verify 24h Before Presentation):

- **1. Contributor Graph:** Insights > Contributors displays all 3 team members with healthy, proportional commit bars (~33% each).
- **2. Network Tree:** Insights > Network shows feature branches branching off main and merging cleanly via Pull Requests.
- **3. Closed PR History:** Pull Requests > Closed shows 10-15 closed PRs, each reviewed and approved by teammates.
- **4. Branch Protection:** Settings > Branches displays "main is a protected branch" badge with approval enforcement.
- **5. Commit Footprints:** Commit history contains small, scoped Conventional Commits with zero generic "final" or "upload" commits.
- **6. Security Hygiene:** Zero credentials committed; backend/.env.example exists; .env is properly gitignored.
- **7. Tree Cleanliness:** Repository contains no node_modules/ or build artifacts.